

Lab #6

Group number : 5

Group members : Aditya Ray - 1233750715

Krutyanjay Shinde - 1234108189

PRE LAB

1. Read through the subsequent sections of this lab and state all of the requirements that your robot program must satisfy.
 - The robot should be able to :-
 1. **Climb** the provided ramp, (15 degree slope).
 2. Detect when it has reached the **top flat surface** and stop.
 3. Turn around (180 degrees).
 4. Go **back down** the ramp (all while not falling off the edge).
 - Safety features robot must have :-
 1. It must use its **line sensors** to continuously detect the dark-colored edges.
 2. When an edge is detected, the robot must take **evasive action**: back up, turn away from the edge, and then resume driving.
 3. While on the slope (both climbing and descending), the robot must use its **accelerometer and gyroscope** (specifically the **roll** measurement) to drive straight up or straight down.

2. Review [Section 6.5, Line and bump sensors](#) of the [Pololu 3pi+ 2040 robot User's Guide](#). How do the line sensors work? Is it possible to use the line sensors in combination with the bump sensors?
 - The line sensors used in the Pololu 3pi+ 2040 are RC-type QTR reflectance sensors. They work by **turning on the IR emitters**, briefly **driving each sensor line HIGH** to precharge the node, then **releasing the line** and **measuring how long it takes to fall LOW**. The fall time depends on the amount of IR light reflected from the surface: if the surface is more reflective (near white), the time will be very short; if it is darker (toward black), the time will be longer.
 - Reflectance also depends on the material, environment, and distance from the sensor, so you should **calibrate the sensors first** and set appropriate high and low thresholds.
 - Technically, yes. Because the line and bump sensors use different GPIO pins and have their own circuits, they can be used together. However, as the Pololu user guide points out, **activating both emitter groups at the same time can cause interference**, so it's best **not to read both sets simultaneously**—enable one group, take a reading, then disable it before reading the other.

3. Recall from the [Sensors](#) lab that you used an accelerometer to measure pitch and roll of the robot. Assume you have a measurement p of pitch and r of roll. If the robot is sitting on a ramp and its x axis is pointing straight down the hill, then what values should you see for r ? What should the sign of p be (assuming the angle is given in the range $-\pi / 2$ to $\pi / 2$)?
 - The value for roll should be 0. This is because roll measures the angle of the robot in a side-to-side direction; in simpler words, it indicates if one of the tires is higher than the other. Since our ramp surface is even, the roll will be 0.
 - The sign of pitch should be negative.

Line Sensing

1. [Examine and run the provided program src/LineDisplay.lf. How does this work?](#)
 - The program works by switching between two modes: calibration and measurement.
 - First, it spends 10 seconds in calibration mode. Because the Line reactor is telling the sensors to learn the min and max values (light and dark) instead of outputting data, the screen only shows a timer counting up.
 - After 10 seconds, it switches to measurement mode. And since the calibration is done, the sensors start sending normalized readings (0 to 1000) to the display reactor, which updates the screen with the sensor values every 100ms.

Hill Climbing

Postlab

1. [Encoders report the angle of the wheels in degrees relative to some starting point. Explain how the `MotorsWithFeedback` reactor uses this data to estimate the speed at which the robot is actually traveling.](#)
 - The `MotorsWithFeedback` reactor estimates speed by calculating the distance the wheel traveled and dividing it by the time it took to travel that distance.
 - It calculates distance by looking at the encoders. Because the encoders report "ticks" (steps of rotation), the code subtracts the previous tick count from the current one to find the change. It then multiplies this change by a conversion factor (derived from the wheel diameter) to turn those ticks into actual meters.
 - It calculates time by tracking the interval between reactor triggers. It takes the current logical time, subtracts the time from the last execution, and converts that duration from nanoseconds into seconds.

- Finally, it divides the calculated meters by the calculated seconds using this formula found in the code:
- $\text{Speed} = \text{Time Interval (seconds)} (\text{Current Ticks} - \text{Previous Ticks}) \times \text{TICKS_PER_METER}$
- *Even though the code names the constant `TICKS_PER_METER`, it is mathematically calculated as **Meters Per Tick**, which is why we multiply it by the ticks to get the distance in meters.*

2. What were your takeaways from the lab? What did you learn during the lab? Did any results in the lab surprise you?

- We learned how to use the line sensors to detect the edges of our calibrated surface (if the edges are dark). The line sensors also help prevent the robot from driving off a high surface because, just like a dark line, we won't get any reflection back from the cliff (given sufficient height).
- Additionally, the bump sensors help us detect any obstacles that might be in our way.
- We also learned how to use the motor encoders to implement control logic that keeps the speed of the robot constant.